

METODOLOGÍA I

Trabajo Práctico — De tu primera spec al panorama de herramientas de SDD

OBJETIVO GENERAL

Elegir una feature propia y mínima, y recorrerla de punta a punta por las dos mitades de esta unidad: primero redactar a mano —sin usar IA para escribirlos— un `constitution.md` y un `feature-spec.md`, entregarlos a un agente para que la implemente, y validar la implementación criterio por criterio contra el comportamiento real, ajustando la especificación (nunca el código directamente) ante cualquier criterio que falle. Después, tomar ESA MISMA feature y recorrerla a través del panorama de herramientas de SDD: reescribir un criterio en notación EARS (Kiro), ubicar dónde encajaría el paso `clarify` de GitHub Spec Kit, mapear cómo se resolvería con las personas-agente de BMAD-METHOD, y completar la tabla comparativa de las cuatro herramientas aplicada a tu propio caso, cerrando con una justificación por escrito de por qué seguir con OpenSpec/OPSX el resto de la cursada.

MARCO TEÓRICO

Documento / práctica	Qué contiene
constitution.md	Reglas fijas del proyecto: lenguaje, estilo, qué el agente puede o no puede hacer sin aprobación.
feature-spec.md	Criterios de aceptación de la feature puntual, cada uno verificable V/F, con al menos un criterio funcional y uno no funcional.
Validación	Se prueba cada criterio contra el comportamiento real de la implementación, nunca "a ojo" leyendo el código.
Ajuste	Si un criterio falla, se corrige la especificación (no el código a mano) y se vuelve a ejecutar.

Herramienta	Rasgo distintivo	Ciclo
Kiro (AWS)	Notación EARS: máximo formalismo. Flujo requirements.md → design.md → tasks.md, con ejecución concurrente de tareas independientes.	Lineal
GitHub Spec Kit	Paso clarify explícito, entre specify y plan. Flujo constitution → specify → clarify → plan → tasks → implement.	Lineal

Herramienta	Rasgo distintivo	Ciclo
BMAD-METHOD	Personas-agente especializadas (Product Owner, Arquitecto, Desarrollador, Tester) colaborando sobre los mismos artefactos.	Re-entrante
OpenSpec / OPSX	Acciones no bloqueantes explore → propose → apply → archive, revisitables en cualquier momento.	Re-entrante

Las cuatro herramientas comparten la misma columna vertebral —especificación → diseño → tareas → implementación → validación— y difieren en el grado de formalismo y en si el ciclo es lineal o re-entrante. Esa columna vertebral común es la que vas a usar como criterio en la segunda mitad de este trabajo práctico.

PARTE A — REDACCIÓN DE LA SPEC

Elegí una feature mínima y propia (distinta a la de "lista de tareas" vista en las Actividades) — puede ser parte de un TP anterior o una idea nueva, pero tiene que ser chica: algo resoluble en pocas horas de desarrollo. Vas a usar esta MISMA feature durante todo el trabajo práctico, incluida la segunda mitad.

1. constitution.md

Redactá un mínimo de 5 reglas fijas para el proyecto donde vive esta feature (lenguaje, estilo, qué no puede hacer el agente sin aprobación, manejo de datos, etc.).

2. feature-spec.md

Redactá un mínimo de 4 criterios de aceptación FUNCIONALES y 2 NO FUNCIONALES, cada uno numerado (CA-1, CA-2, ...) y verificable V/F contra una implementación concreta.

PARTE B — EJECUCIÓN Y VALIDACIÓN

Entregale ambos documentos a un agente (Claude Code, Antigravity u otro) y pedile que implemente la feature a partir de ellos, sin agregar instrucciones adicionales fuera de lo escrito en la spec.

1. Tabla de validación

Por cada criterio de aceptación, documentá: el código del criterio, cómo lo probaste concretamente (no "lo revisé", sino la acción real que hiciste), y el resultado (V o F).

2. Ajuste de la spec

Si algún criterio dio F, mostrará la versión ANTERIOR y la versión AJUSTADA de ese criterio en la spec (no un parche en el código), y volvé a validarlo. Si todos dieron V a la primera, elegí el criterio que más te costó redactar bien y explicá qué versión previa, menos precisa, habías descartado antes de llegar a la definitiva.

PARTE C — SPEC VS. PROMPT LARGO

Reflexión breve (mínimo 100 palabras): ¿qué hubiera pasado si, en vez de escribir estos dos documentos, le hubieras dado al agente un único prompt largo con la misma información? Explicá con tus propias palabras qué perdés al no tener una especificación que persiste, se versiona y se audita. Este mismo patrón (spec → ejecución → validación) no es invención de la cátedra: en la segunda mitad de este trabajo práctico vas a ver que varias herramientas reales de la industria lo sostienen, cada una a su manera.

PARTE D — TU CRITERIO EN NOTACIÓN EARS

Elegí uno de los criterios de aceptación del `feature-spec.md` que redactaste en la Parte A.

1. Criterio original

Transcribí el criterio tal como lo tenías redactado, en lenguaje libre.

2. Reescritura en EARS

Reescribilo siguiendo el patrón `WHEN [evento] THE SYSTEM SHALL [comportamiento]`. Si el criterio original ya era ambiguo, la reescritura en EARS debería obligarte a precisar el evento disparador exacto y el comportamiento observable exacto.

3. Reflexión breve

¿Qué ganaste en precisión al pasar a EARS? ¿Qué perdiste en naturalidad de lectura? ¿Este criterio en particular se beneficia del formalismo de EARS, o el costo de rigidez no se justifica para tu caso?

PARTE E — EL PASO CLARIFY SOBRE TU CASO

Ubicá, sobre el mismo criterio de la Parte D, dónde encajaría el paso `clarify` de GitHub Spec Kit si tu proyecto siguiera ese flujo.

1. Preguntas de aclaración

Escribí al menos dos preguntas concretas que un agente debería hacerte ANTES de planificar la implementación de ese criterio —preguntas sobre huecos reales que tu especificación original dejaba sin resolver (por ejemplo: límites de longitud, comportamiento ante datos inválidos, casos de borde no contemplados).

2. Por qué importa

Explicá con tus propias palabras por qué esas preguntas, si no se hacen antes de planificar, terminarían resueltas arbitrariamente por el agente durante la implementación —conectando con el problema de "rellenar huecos" que venimos arrastrando desde el comienzo de esta unidad.

PARTE F — MAPEO CON LAS PERSONAS-AGENTE DE BMAD

Mapeá cómo se vería resuelta tu feature si tu proyecto siguiera el patrón de personas-agente de BMAD-METHOD (Product Owner, Arquitecto, Desarrollador, Tester).

1. Qué persona depende de qué artefacto

Para tu caso concreto, completá la cadena: qué produciría el/la **Product Owner/Analista** (la spec de tu feature), qué necesitaría el/la **Arquitecto** de esa spec para producir un diseño, qué necesitaría el/la **Desarrollador/a** de ese diseño para producir el código, y qué necesitaría el/la **Tester** de ese código para producir la validación.

2. Relación con la Unidad 5

Explicá en qué se parece este mapeo al patrón de sub-agentes con DAG de dependencias que viste en la Unidad 5 (Agentes especializados y orquestación), y qué pasaría en tu caso si alguna de las cuatro personas-agente intentara trabajar sin que la anterior haya entregado su artefacto.

PARTE G — TABLA COMPARATIVA APLICADA A TU CASO Y JUSTIFICACIÓN

1. Tabla comparativa

Completá la siguiente tabla aplicada específicamente a tu feature (no en abstracto): para cada herramienta, indicá cómo se vería tu criterio de la Parte D si lo hubieras especificado con esa herramienta, y qué ganarías o perderías respecto de cómo lo hiciste en la Parte A con OpenSpec.

Herramienta	Cómo se vería tu criterio con esta herramienta	Qué ganás / qué perdés respecto de tu spec de la Parte A
Kiro (EARS)		
GitHub Spec Kit		
BMAD-METHOD		
OpenSpec / OPSX		

2. Justificación por escrito (mínimo 120 palabras)

Con tu propio caso como evidencia, justificá por escrito por qué vas a seguir usando OpenSpec/OPSX el resto de la cursada. Tu justificación tiene que apoyarse en algo concreto de tu feature (no solo repetir el argumento general de la clase): por ejemplo, un momento en que tu spec de la Parte B tuvo que ajustarse a mitad de camino, y cómo el modelo no bloqueante de OPSX lo hizo posible sin romper el proceso.

CONSIDERACIONES

- ✓ **Una sola feature, de punta a punta:** el TP no pide una feature distinta por parte — se elige UNA en la Parte A y se recorre por las siete partes hasta la G.
- ✓ **Elección de la feature:** tiene que ser chica y acotada — el TP evalúa la calidad de la spec y de la validación, no la complejidad de la feature.
- ✓ **Constitution.md real:** reglas vagas tipo "código limpio" no cuentan; tienen que ser accionables (negativas o condicionales).
- ✓ **Criterios verificables:** un criterio de aceptación que no pueda responderse V/F contra la implementación se considera incompleto.
- ✓ **Validación real, no a ojo:** "revisé el código y parece que anda" no es una validación válida; hace falta la acción concreta que comprueba el criterio.
- ✓ **EARS bien aplicado:** la reescritura de la Parte D tiene que seguir estrictamente el patrón WHEN...THE SYSTEM SHALL, no una paráfrasis libre con esas palabras sueltas.
- ✓ **Preguntas de clarify reales:** las preguntas de la Parte E tienen que apuntar a huecos genuinos de tu especificación original, no preguntas genéricas que aplicarían a cualquier feature.
- ✓ **Tabla completa, no en abstracto:** la tabla de la Parte G se completa pensando en tu feature puntual, no copiando la tabla general de la clase.

CONCLUSIONES ESPERADAS

- Redactar constitution.md y feature-spec.md propios, distinguiendo reglas transversales de criterios de una feature puntual.
- Validar una implementación criterio por criterio, nunca "a ojo", contra el comportamiento real.
- Practicar el hábito central del cuatrimestre: cuando algo falla, se corrige la spec, no el código.
- Reescribir un criterio propio en notación EARS y evaluar cuándo se justifica ese formalismo.
- Ubicar preguntas de aclaración reales que el paso clarify resolvería antes de planificar.
- Mapear el flujo de personas-agente de BMAD sobre un caso concreto, reconociendo el mismo DAG de dependencias de la Unidad 5.
- Argumentar con evidencia propia —no solo con el argumento general de la clase— por qué OpenSpec/OPSX es la herramienta elegida para el resto de la cursada.